
Integrative Programming and Technologies III

Test 2 (Design patterns, XML and data exchange, and Network programming)

Total: **35 marks**

19 May 2023 Time: 1 hour

Questions 1-7

7 marks

1. Distributed applications architecture makes it easier to add new components.
2. XML documents must be encoded for data exchange between different platforms.
3. DTD translates how the data in an XML document should be displayed in a web page.
4. The RMI software hides the low-level communication between the client and server from the programmer.
5. In the web service architecture, it is objects that are published in the registry.
6. In a client-server application, it is the client that listens for requests from the server.
7. RMI is based on the web services architecture. (F)

Questions 8-9

4 marks

1. “Distributed applications consist of independent processes”. Briefly explain what is an **independent process**. (2 marks)
2. Why is exception handling an important part of distributed applications? (2 marks)

Question 10

6 marks

The class below shows an implementation of the **Singleton** pattern.

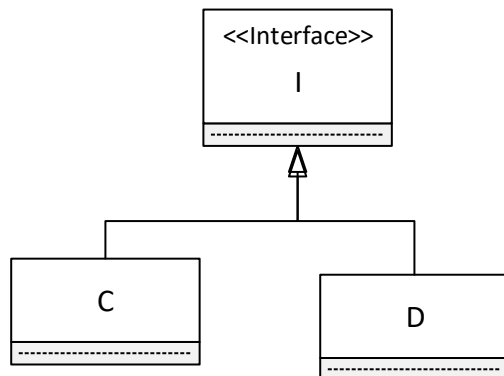
```
public class Printer {
    private static Printer p;
    private Printer() {

    }
    public static Printer getInstance() {
        if(p==null) {
            p=new Printer();
        }
        return p;
    }
}
```

- 1.1. Explain why the method `getInstance()` is declared static. (2 marks)
- 1.2. The following code creates an instance of `Printer`. State whether the statement is **valid** or **invalid**. Explain your answer. (2 marks)

```
public class Tester {
    public static void main(String[] args) {
        Printer p1=new Printer();
        ....
    }
}
```

2. The classes **C** and **D** implement the interface **I**. You want to create instances of the subclasses. Which design pattern provides guidelines. Explain your answer. (2 marks)



Question 11

10 marks

Below is part of an XML document (**phonebook.xml**). Each **Contact** has **id** (a unique identifier), **name**, **phone** number and **address** (which consists of **street** and **city**).

```
.....
<PhoneBook>
    <Contact id="1">
        <name>K Lo</name>
        <phone>046 222 2222</phone>
        <address>
            <street>11 Osmond Terrace</street>
            <city>Grahamstown</city>
        </address>
    </Contact>
    .....
</PhoneBook>
```

1. Complete the DTD document (**phonebook.dtd**) below, which defines the structure of the data in **phonebook.xml**. (6 marks)

```
<?xml version="1.0" encoding="UTF-8"?>
<!ELEMENT PhoneBook (Contact)+>
<!ELEMENT Contact (name,phone,address)>
<!-- id --> (2 marks)
<!ELEMENT name (#PCDATA)>
<!ELEMENT phone (#PCDATA)>
_____ (2 marks)
_____ (1 mark)
_____ (1 marks)
```

2. Complete the tag below that links the DTD to the XML document. (2 marks)

```
<!DOCTYPE _____ SYSTEM _____>
```

3. Below is part of the code to retrieve the **Contact** objects in the XML document. It creates an instance of **Contact** and calls **setId(String)** to set the value of the “**id**” attribute. Assume all variables have been declared. If the instance of **XMLStreamReader** is **xmlreader**, complete the missing code to retrieve and set the value of “**id**”. (2 marks)

```
.....
case XMLStreamConstants.START_ELEMENT:{
    element=xmlreader.getLocalName();
    if(element.equals("Contact")) {
        Contact contact=new Contact();
        contact.setId(_____);
    }
}
```

Question 12

8 marks

Below is the class **ServerMark**, a server-side program.

```
ServerMark
  ● response() : void
  ● status(double) : String
```

The method **status(double)** takes a **mark** (i.e., final mark, double) and returns the results (e.g., “Failed”, “Passed”, or “Distinction”). The method **response()** reads **studId** (student id, a string) and **mark** (final mark, double) from a socket, calls **status(double)** and writes back the **studId**, **mark** and **results** (e.g., Student ID=202020 Final mark=70.0 Status =Passed) to the socket.

Complete the missing parts of **response()** below.

```
public void response() throws Exception{
    ServerSocket server=new ServerSocket(1989);
    Socket socket=server.accept();
    _____
    _____
    _____
    _____
    server.close();
}
```